Helmet Detection & Road Safety Analyzer

Deployment Architecture Reference · Python 3.12 · YOLOv8 8.4.182 · Streamlit 1.66.0 · ByteTrack · Streamlit Community Cloud (intended)

1. Executive Overview

Road safety enforcement faces a persistent gap: manual monitoring of helmet compliance is labour-intensive, inconsistent, and non-scalable. This project provides a computer-vision pipeline that identifies motorcyclists and bicyclists, classifies their helmet status in real time, and surfaces violation evidence through a browser-based interface.

- **Single-image analysis** — annotated JPEG + CSV + JSON exports.
- **Batch processing** — multi-image upload; results packaged as a ZIP.
- **Video analysis** — per-frame inference, ByteTrack identity persistence, ROI masking, violation snapshots, timeline CSV/JSON.
- **Live camera (WebRTC)** — real-time inference from browser webcam; violation snapshots throttled to 1 per 4 seconds.
- **Session analytics** — Altair charts for class distribution and confidence histogram; violation archive gallery.

2. System Architecture

Three horizontal layers: developer/source (GitHub), deployment/runtime (Streamlit Community Cloud), and user/browser.

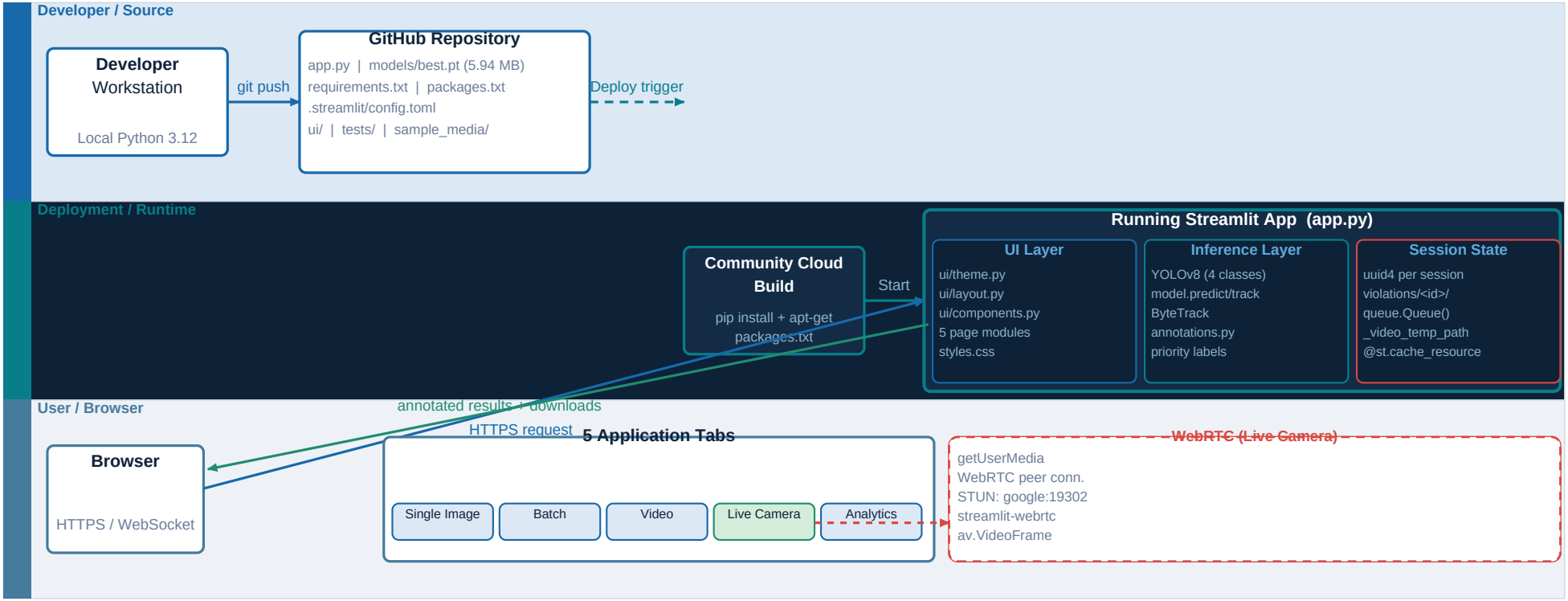

Figure 1 — Three-layer deployment architecture. Dashed arrows = optional/conditional paths (deploy trigger, WebRTC).

3. Input Processing Pipeline

3.1 Single Image

- User uploads a file (JPG / JPEG / PNG / WEBP / BMP, max 15 MB).
- PIL.Image opens the file; converted to a NumPy array.
- model.predict(conf=0.35, iou=0.45) runs synchronously.
- plot_with_priority_labels() renders the annotated frame (helmet/no-helmet priority=2, driver/bicyclist=1; greedy slot nudging, up to 30 iterations).
- Annotated array encoded as JPEG and offered for download.
- Detection data serialised to CSV and JSON.

3.2 Batch Processing

- User uploads multiple files (same format and size limits as single image).
- Per-image predict-and-annotate loop runs sequentially.
- Annotated JPEGs + batch_summary.csv + batch_summary.json written into an in-memory io.BytesIO ZIP archive and offered as a download.

3.3 Video Analysis

- Upload (MP4 / AVI / MOV / MKV, max 100 MB) → NamedTemporaryFile in session violation directory; path stored in session_state._video_temp_path.
- probe_cap reads frame count, FPS, width, height for telemetry banner.
- User configures frame stride (1–5), confidence, IoU, ROI preset, and tracking toggle, then clicks Start.
- Analysis cap iterates frames at stride. Each frame → model.track (persist=True, bytetrack.yaml) or model.predict → plot_with_priority_labels → optional ROI overlay.
- Violation frames: snapshot saved as JPG (max 1/s). Timeline data available as CSV/JSON after analysis. Temp file deleted in finally block.

3.4 Live Camera (WebRTC)

- webrtc_streamer() opens ICE with stun:stun.l.google.com:19302.
- Browser requests camera via getUserMedia; WebRTC peer connection established.
- VideoProcessor.recv() receives each av.VideoFrame, converts to NumPy, runs model.predict(), annotates via plot_with_priority_labels(), returns as av.VideoFrame.
- Violation snapshots saved to session directory, throttled to max 1 per 4 s.

4. Technology Decisions

Technology	Role	Benefit	Trade-off
YOLOv8 (ultralytics 8.4.182)	Object detection	Single-stage; native ByteTrack; simple predict/track API	CPU-only inference; no annotated video download
Streamlit 1.66.0	Web UI framework	Python-native; Community Cloud free tier; no separate frontend	Limited UI customisation; threading constraints
streamlit-webrtc 0.78.1	Browser camera	Handles WebRTC pipeline in Streamlit; no separate media server	Requires webcam; TURN not configured
ByteTrack + lap 0.5.13	Multi-object tracking	Stable id:N labels across video frames	Video mode only; adds lap dependency
PyAV 17.1.0	Video frame codec	Correct colour-space for av.VideoFrame ↔ NumPy	Binary codec dependency
OpenCV headless 5.0.0.93	Frame I/O + ROI	No GUI deps; smaller container footprint	No display functions
Pillow 12.3.0	Image decode/encode	Broad format support for single-image and batch modes	—
torch 2.14.1+cpu	YOLOv8 backend	No GPU required; installs as ultralytics dependency	CPU inference; ~800 MB RAM on free tier
Altair 6.3.0	Analytics charts	Declarative; renders natively in Streamlit; no extra JS bundle	—

5. Deployment Lifecycle

1. Developer writes and tests code locally (Python 3.12.2).
2. Developer pushes to GitHub via git push.
3. Streamlit Community Cloud detects the push and triggers a build.
4. Build installs system packages from packages.txt (libgl1, libglib2.0-0t64).
5. Build installs Python packages from requirements.txt at pinned versions.
6. app.py starts; @st.cache_resource loads models/best.pt once (~5.94 MB).
7. Streamlit serves the app on a public HTTPS URL.
8. Visitor's browser opens the URL; WebSocket-backed connection established.
9. Visitor uses one of the five tabs; inference runs server-side; annotated results returned to the browser.

5.1 Session Isolation

Each session gets a uuid.uuid4().hex subdirectory under violations/ preventing cross-session snapshot leakage. A queue.Queue() per session handles event logging. The YOLOv8 model is shared via @st.cache_resource (read-only inference is thread-safe).

5.2 Operational Considerations

- **Memory** — torch + model consume significant RAM; concurrent video sessions on the ~800 MB free tier can be memory-intensive.
- **WebRTC** — STUN resolves ICE for most residential networks. Corporate firewalls blocking UDP may prevent connection. No TURN configured.
- **Ephemeral storage** — violation snapshots and temp video files lost on process restart. Persistent storage requires an external object store.
- **File limits** — images 15 MB (JPG/PNG/WEBP/BMP); video 100 MB (MP4/AVI/MOV/MKV).

6. Test Coverage

Test Class	Tests	What is verified
TestHelmetDetectionPipeline	6	Model loads (4-class map) · single-image inference · ByteTrack assigns IDs · ROI filtering · file-type validation · batch ZIP contents
TestLabelPlacement	9	Priority order (helmet/no-helmet=2, driver/bicyclist=1) · overlap detection · boundary clamping · box immutability · labels-off path
TestTrackingIdLabels	9	id:N prefix per class with tracking · distinct IDs · no prefix without tracking · show_conf=False keeps id: · render shape · coord immutability
Total	24	All pass in approximately 2.5 seconds

7. Known Limitations

- No annotated video download — CSV/JSON timeline only.
- Live camera requires a physical webcam; non-functional in headless environments.
- Violation archive is session-scoped (violations//) and lost on process restart.
- CPU inference only on Community Cloud free tier.
- Compliance rate = helmets ÷ (helmets + no-helmets); driver/bicyclist counts excluded.
- ~800 MB free RAM on Community Cloud; video + model inference can approach this.
- WebRTC STUN fails behind strict corporate firewalls; TURN not configured.

8. STAR Interview Summary

	Summary
Situation	Road safety monitoring relies on manual observers — inconsistent and non-scalable. A CV system was needed to detect helmet compliance from image, video, and live-camera inputs at low cost.
Task	Build a full-stack detection app: fine-tuned YOLOv8 (4 classes), Streamlit web UI with 5 tabs, multi-session isolation, a custom priority label renderer, and a 24-test pytest suite.
Action	Selected YOLOv8 for native ByteTrack integration. Built plot_with_priority_labels() with two-tier priority and greedy slot nudging (up to 30 iterations). Per-session UUID directories isolate violation snapshots. Targeted Streamlit Community Cloud to avoid server-management overhead.
Result	Working 5-tab application (image, batch, video, live-camera, analytics). 24 tests pass in ~2.5 s. No database, no REST API, no Docker. Primary constraint: live-camera tab requires a webcam and a WebRTC-permitting network.